

SOR iteration for sparse matrices

Ožbej Kresal

June 2026

In this exercise we implemented a sparse matrix datatype `RedkaMatrika` and the necessary interfaces to use the datatype to solve a $Ax = b$ system using SOR iteration. We then tested our sparse matrix and SOR implementation on two graphs, where we used the physical method for graph embedding to embed them in a plane.

1 Sparse matrix implementation

We defined a new data type `RedkaMatrika`, which stores the non-zero elements of the matrix due to space requirements in two double arrays `V` and `I`, such that:

$$V[i][j] = a_{i, I[i][j]}$$

Therefore, the array `V` contains the non-zero elements of the matrix `A`, whereas the array `I` contains the column indices. The array `V[i]` thus contains the non-zero elements from the i -th row of matrix `A`, and the array `I[i]` stores the column indices of these non-zero elements in the same order.

Example:

$$A = \begin{bmatrix} 10 & 0 & 2 \\ 0 & 5 & 0 \\ 3 & 0 & 8 \end{bmatrix}, \quad V = \begin{bmatrix} 10 & 2 \\ 5 & \\ 3 & 8 \end{bmatrix}, \quad I = \begin{bmatrix} 1 & 3 \\ 2 & \\ 1 & 3 \end{bmatrix}.$$

1.1 Operations

We then implemented the following operations:

- **Base.size** Returns the dimensions of the sparse matrix.
- **Base.getindex** Returns the value at position `(i,j)`. If the element is not stored, it is assumed to be zero.
- **Base.setindex!** Updates the value at position `(i,j)` and modifies the sparse representation accordingly.
- **Base.*** Implements matrix vector multiplication using only the stored non-zero elements, making the operation more efficient than with a dense matrix.

1.2 SOR iteration to solve sparse linear systems

To solve sparse systems $Ax = b$ we implemented a SOR iteration function adapted to work with our sparse matrix datatype:

$$x, it = \text{sor}(A, b, x_0, \omega, \text{tol} = 10^{-10}),$$

where x_0 is the initial approximation, `tol` is the condition for stopping the iteration, and ω is the parameter in the SOR iteration. The iteration then stops when

$$\|Ax^{(k)} - b\|_{\infty} < \delta,$$

where δ is given by the `tol` argument.

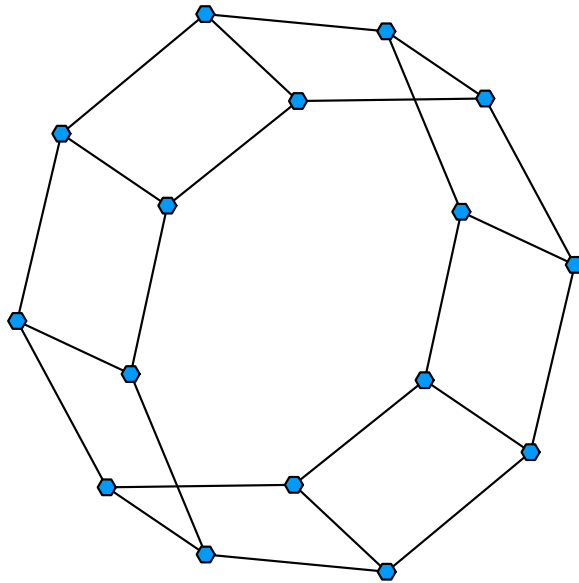
By adapting the method to our sparse matrix implementation, we effectively reduce the computational cost of a single iteration from $\mathcal{O}(n^2)$ to $\mathcal{O}(\text{number of non-zero elements})$, which greatly increases the speed of computation for sparse matrices.

2 Demonstration - embedding graphs

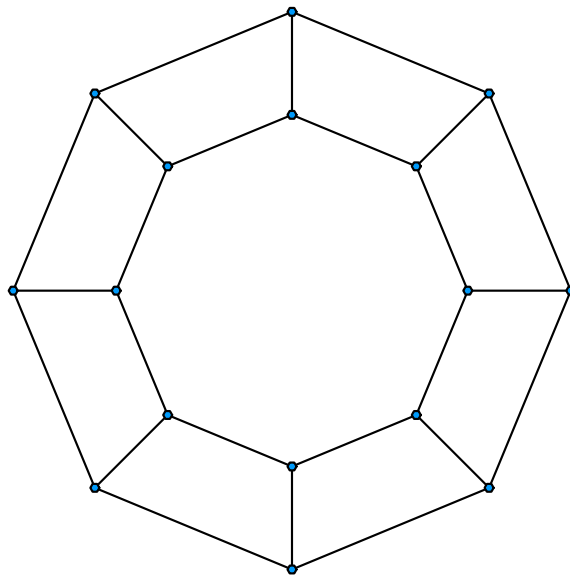
To demonstrate our implementation we will embed two graphs in a plane using the physical method, which we adapted to work with our sparse matrix SOR implementation.

2.1 Cyclic ladder graph

Unembedded graph:

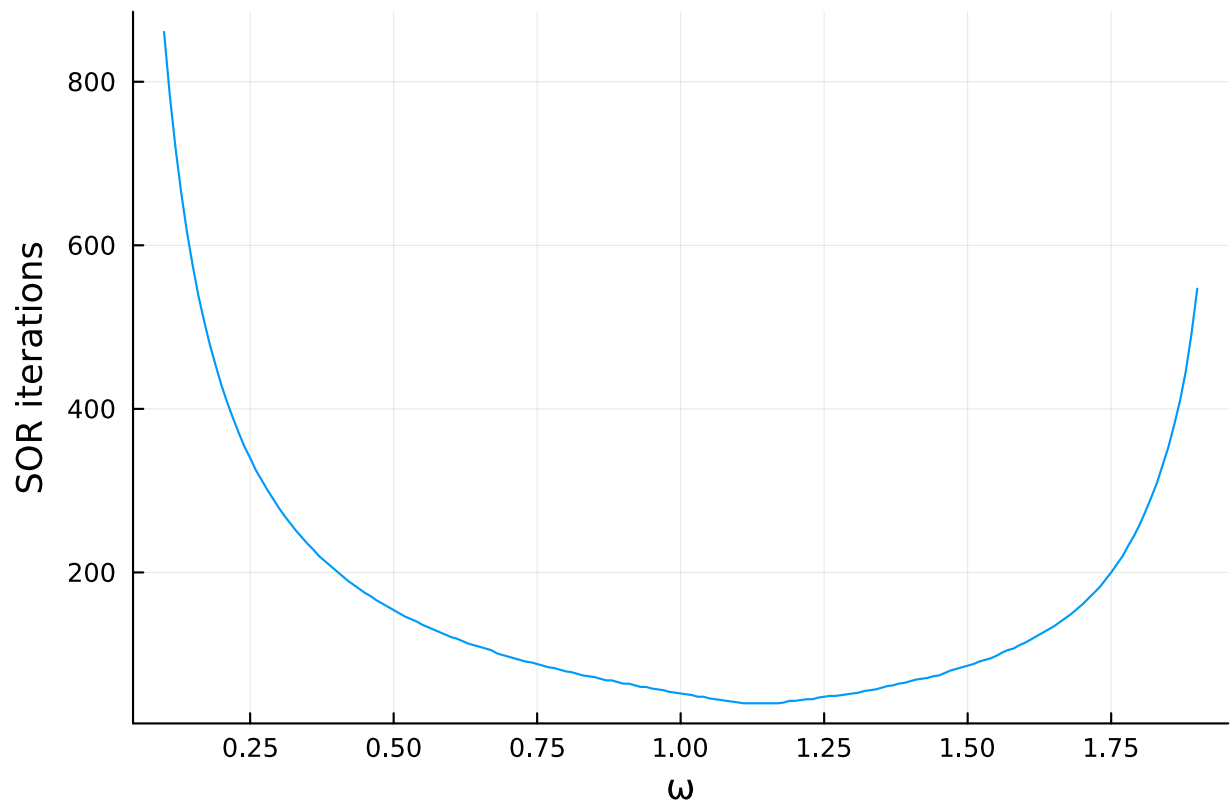


Embedded graph using the physical method with our SOR implementation:



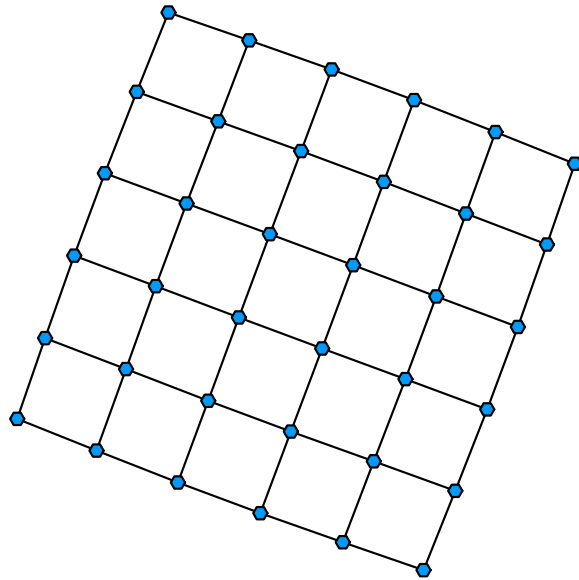
Effect of ω on SOR convergence:

Best omega for ladder: 1.11

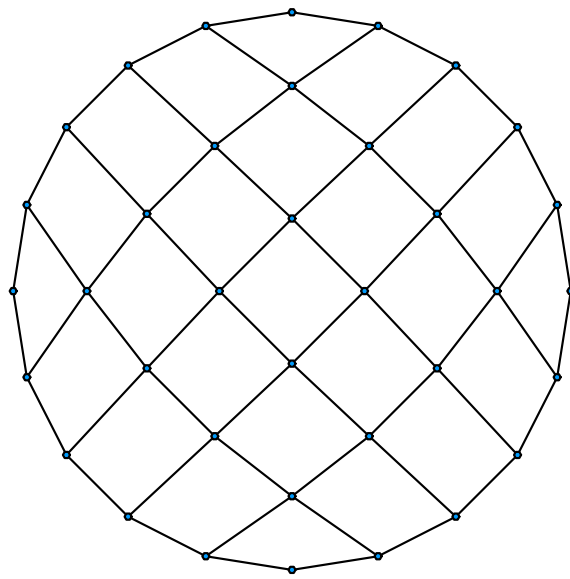


2.2 Grid graph

Unembedded graph:



Embedded graph using the physical method with our SOR implementation, with some points fixed in a circle:



Effect of ω on SOR convergence:

Best omega for grid: 1.26

